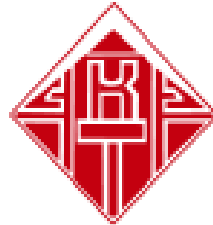


TRƯỜNG ĐẠI HỌC KIẾN TRÚC ĐÀ NẴNG
KHOA CÔNG NGHỆ THÔNG TIN



TRƯỜNG ĐẠI HỌC KIẾN TRÚC ĐÀ NẴNG
DANANG ARCHITECTURE UNIVERSITY

ĐỀ TÀI:

Hệ thống Quản Lý Điểm Danh

Học phần : Chuyên đề công nghệ mới

Giảng viên hướng dẫn : Lâm Tùng Giang

Lớp : 22CT3

Nhóm sinh viên thực hiện :

Nguyễn Phan Phú Hoàng	2251220223
-----------------------	------------

Đàm Phạm Tiến	2251220209
---------------	------------

Dương Nguyễn Duy Phước	2251220238
------------------------	------------

Cao Xuân Đạt	2251220231
--------------	------------

Chung Văn Long	2251220127
----------------	------------

Đà Nẵng, ngày tháng năm 2026

MỤC LỤC

CHƯƠNG 1: TỔNG QUAN	3
1.1 Mục tiêu.....	3
1.2 Mô tả chức năng.....	3
1.3 Phạm vi.....	4
CHƯƠNG 2: THIẾT KẾ HỆ THỐNG	6
2.1 Kiến trúc tổng thể.....	6
2.2 Kiến trúc DevOps.....	7
2.3 API	8
CHƯƠNG 3: TRIỂN KHAI HỆ THỐNG	9
3.1 Backend.....	9
3.2 Frontend	11
3.3 Environment.....	12
3.4 Docker	14
3.6 CI — Continuous Integration.....	17
CHƯƠNG 4: DEPLOY	19
4.1 Môi trường	19
4.2 Quy trình deploy.....	19
4.3 Minh chứng	20
CHƯƠNG 5: LOGGING & DEBUG	20
5.1 Logging	20
5.2 Debug theo layer	20
5.3. INCIDENT	21
CHƯƠNG 6: KẾT QUẢ	25
6.1 Mô tả hệ thống.....	25
6.2 Minh họa	26
6.3 Đánh giá	28
CHƯƠNG 7: KẾT LUẬN.....	29
7.1 Kết quả đạt được	29
7.2 Ưu điểm / Hạn chế	29
Ưu điểm	29
Hạn chế	30

CHƯƠNG 1: TỔNG QUAN

1.1 Mục tiêu

Trong quá trình quản lý sinh viên tại các cơ sở giáo dục, việc điểm danh thủ công thường mất nhiều thời gian, dễ xảy ra sai sót và khó theo dõi lịch sử điểm danh. Để giải quyết vấn đề này, nhóm xây dựng hệ thống **quản lý điểm danh sinh viên** nhằm hỗ trợ quá trình theo dõi chuyên cần, thống kê dữ liệu và quản lý sinh viên một cách hiệu quả.

Mục tiêu của đề tài:

- Xây dựng hệ thống quản lý điểm danh hoàn chỉnh gồm **Frontend + Backend + Database**
- Cung cấp giao diện cho người dùng thao tác dễ dàng
- Cho phép quản lý thông tin sinh viên, lớp học, điểm danh, báo cáo thống kê
- Triển khai hệ thống bằng **Docker**
- Thiết lập **CI/CD** để kiểm tra và triển khai tự động
- Hệ thống có thể chạy trên môi trường production
- Hỗ trợ logging và debug trong quá trình vận hành

Nội dung chuyên môn

Đề tài áp dụng các công nghệ mới trong phát triển phần mềm:

- Frontend hiện đại bằng **React + Vite**
- Backend bằng **Node.js + Express**
- Database **MySQL**
- Container hóa bằng **Docker**
- Tự động kiểm tra bằng **GitHub Actions**
- Quản lý biến môi trường bằng **.env**
- Triển khai theo mô hình **DevOps**

1.2 Mô tả chức năng

Liệt kê chức năng chính (CRUD + trạng thái)

Hệ thống bao gồm các chức năng chính:

Quản lý sinh viên

- Thêm sinh viên
- Sửa thông tin sinh viên
- Xóa sinh viên
- Tìm kiếm sinh viên

- Quản lý trạng thái sinh viên

Quản lý điểm danh

- Điểm danh theo ngày
- Cập nhật trạng thái điểm danh
- Lưu lịch sử điểm danh
- Xem danh sách sinh viên đã điểm danh

Báo cáo thống kê

- Thống kê chuyên cần
- Báo cáo số buổi vắng
- Báo cáo theo khoảng thời gian
- Xuất dữ liệu báo cáo

Quản lý giao diện người dùng

- Trang dashboard
- Quản lý danh sách sinh viên
- Bảng điểm danh
- Báo cáo thống kê
- Kiểm tra sức khỏe hệ thống

Chức năng hệ thống

- API backend
- Logging request
- Health check
- Docker deploy
- CI pipeline
- Quản lý cấu hình qua environment

1.3 Phạm vi

Bao gồm

- Quản lý sinh viên
- Quản lý điểm danh
- Báo cáo thống kê
- Frontend giao diện web
- Backend REST API
- Kết nối database MySQL
- Docker container

- CI/CD pipeline
- Logging hệ thống
- Health check API

Không bao gồm

- Mobile application
- Tích hợp AI nhận diện khuôn mặt
- Xác thực bằng OTP
- Thanh toán online
- Tích hợp email notification
- Phân tán đa server

CHƯƠNG 2: THIẾT KẾ HỆ THỐNG

2.1 Kiến trúc tổng thể

Hệ thống được thiết kế theo mô hình **3 lớp (3-tier architecture)**:

1. **Presentation Layer (Frontend)**

Cung cấp giao diện người dùng bằng React, hiển thị dữ liệu và gửi request tới backend.

2. **Business Logic Layer (Backend)**

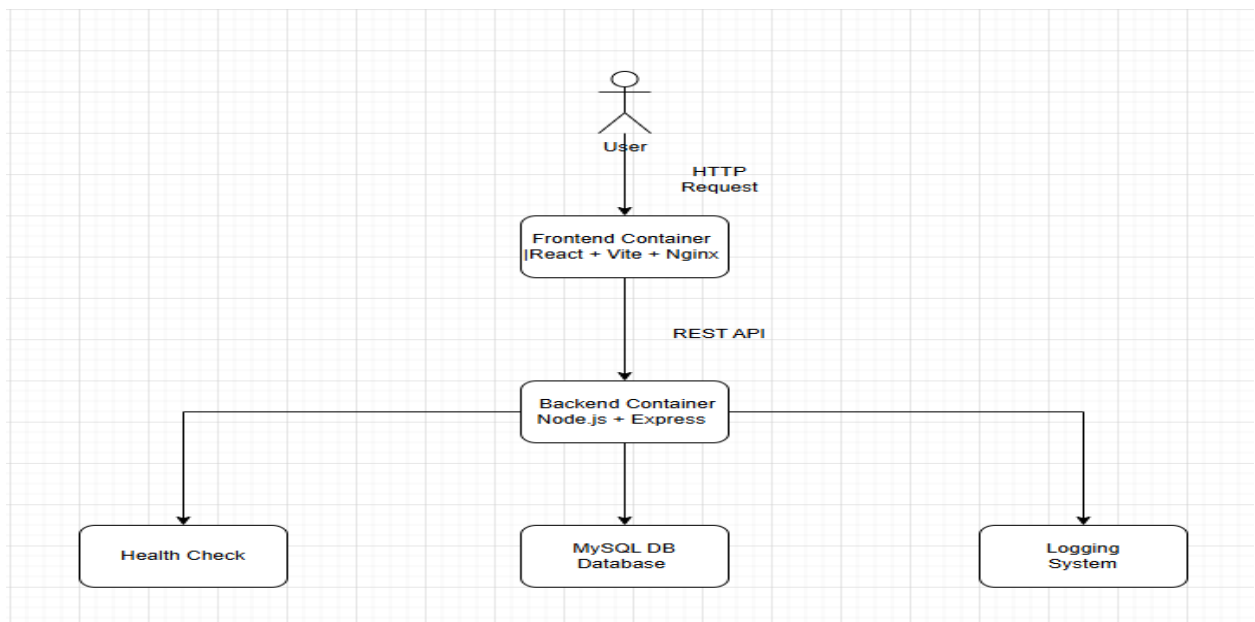
Xử lý nghiệp vụ, nhận request từ frontend, thao tác với database và trả dữ liệu.

3. **Data Layer (Database)**

Lưu trữ dữ liệu sinh viên, điểm danh, báo cáo thống kê.

Luồng hoạt động

User → Frontend → Backend API → Database → Backend → Frontend → User



Hình 1: Sơ đồ kiến trúc tổng thể hệ thống

Giải thích kiến trúc

- Người dùng truy cập giao diện web
- Frontend gửi request đến backend thông qua REST API

- Backend xử lý logic nghiệp vụ
- Backend kết nối MySQL để đọc/ghi dữ liệu
- Dữ liệu được trả lại frontend để hiển thị

Kiến trúc này giúp:

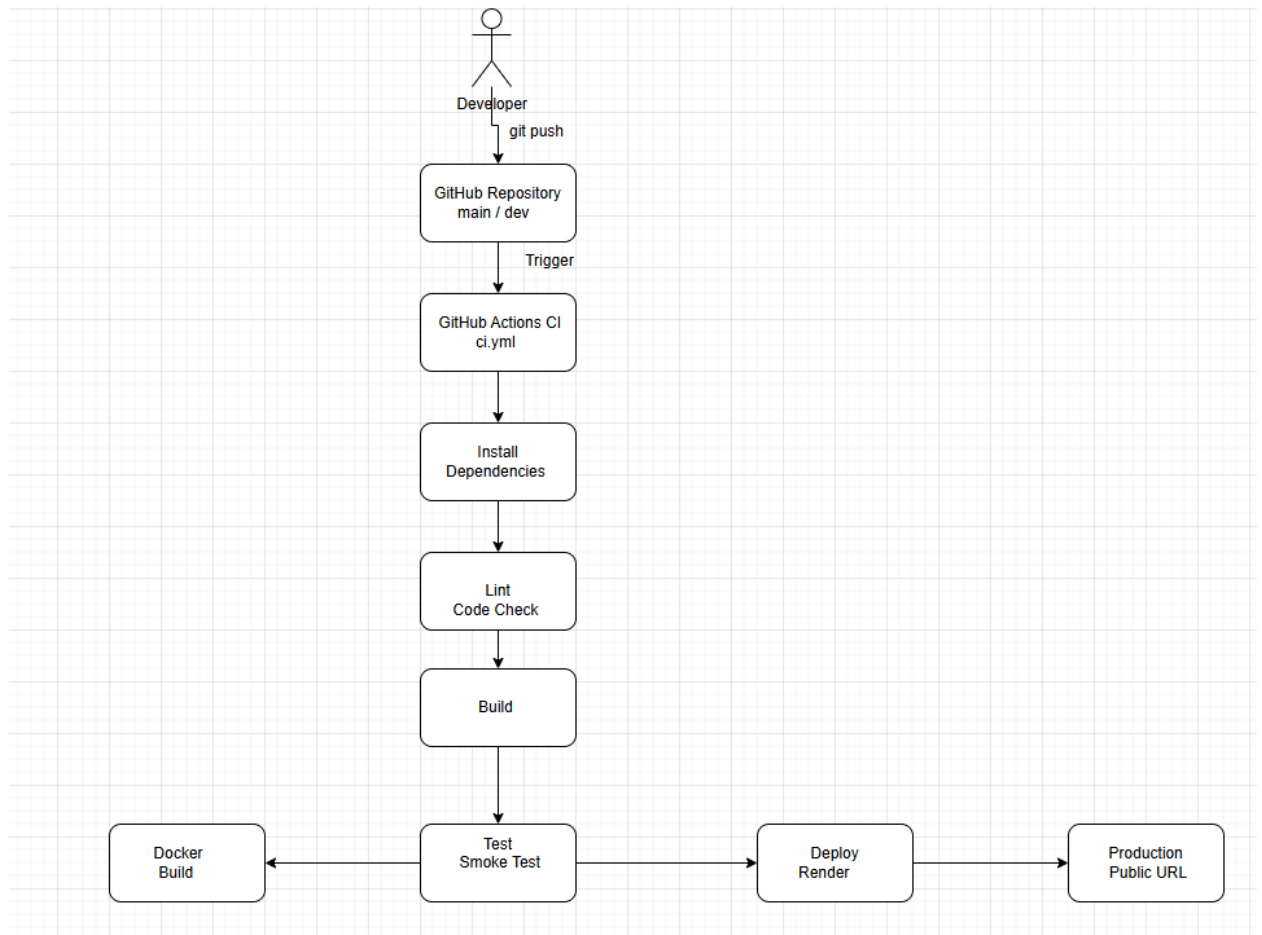
- Dễ bảo trì
- Dễ mở rộng
- Tách biệt frontend/backend
- Phù hợp với triển khai Docker

2.2 Kiến trúc DevOps

Hệ thống được xây dựng theo quy trình DevOps để tự động hóa kiểm tra và triển khai.

Quy trình DevOps

1. Developer viết code
2. Push code lên GitHub
3. GitHub Actions chạy pipeline CI
4. Kiểm tra lint / build / test
5. Build Docker image
6. Deploy lên môi trường chạy



Hình 2: Sơ đồ quy trình CI/CD của hệ thống

Flow DevOps

Code → GitHub → CI Pipeline → Build → Docker → Deploy

2.3 API

Hệ thống sử dụng kiến trúc **RESTful API**

Health Check API

+ GET /api/health

Chức năng:

- Kiểm tra backend đang hoạt động
- Kiểm tra trạng thái hệ thống

Một số API chính

Sinh viên

GET /api/students

POST /api/students

PUT /api/students/:id

DELETE /api/students/:id

Chức năng:

- Lấy danh sách sinh viên
- Thêm sinh viên
- Cập nhật thông tin
- Xóa sinh viên

Điểm danh

GET /api/attendance

POST /api/attendance

PUT /api/attendance/:id

Chức năng:

- Lấy dữ liệu điểm danh
- Thêm dữ liệu điểm danh
- Cập nhật điểm danh

Báo cáo

GET /api/reports

GET /api/stats

Chức năng:

- Lấy dữ liệu báo cáo
- Thống kê chuyên cần

CHƯƠNG 3: TRIỂN KHAI HỆ THỐNG

3.1 Backend

Backend được xây dựng bằng:

- Express.js
- Node.js

Chức năng backend

- Xử lý request từ frontend
- Xử lý business logic
- Truy vấn database
- Trả dữ liệu JSON
- Logging request
- Health check

Cấu trúc backend

- routes/ : định nghĩa API
- controllers/ : xử lý logic
- models/ : thao tác dữ liệu
- middleware/ : xử lý trung gian
- config/ : cấu hình hệ thống

Kết nối database

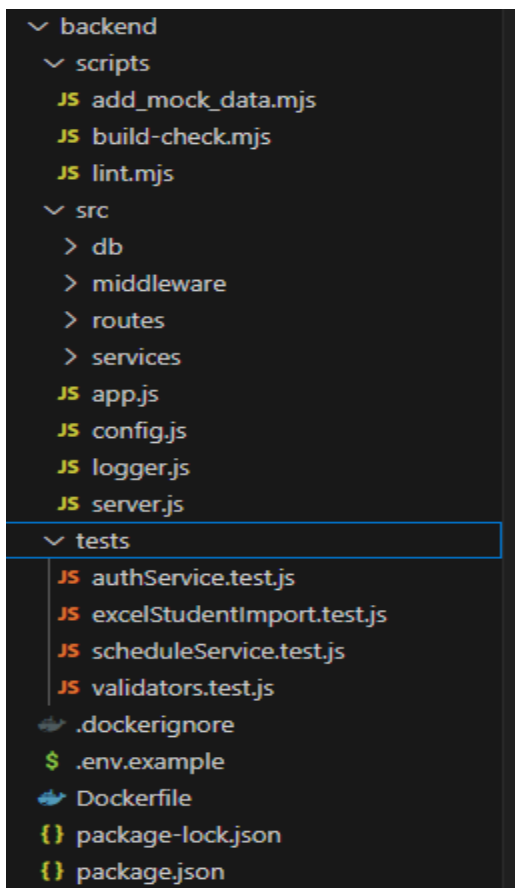
Backend sử dụng:

- MySQL 8.4

Thông qua cấu hình:

- DB_HOST
- DB_PORT
- DB_USER
- DB_PASSWORD
- DB_NAME

Backend lấy cấu hình qua file .env



Hình 4: Cấu trúc thư mục backend

3.2 Frontend

Bắt buộc:

Công nghệ

- React.js
- Vite
- Axios
- React Router DOM
- ESLint

Gọi API backend:

```
frontend > src > api > JS client.js > ...
1  const API_BASE_URL = import.meta.env.VITE_API_BASE_URL || "/api";
2  const SESSION_KEY = "attendance.session";
3
4  export function getStoredSession() {
5    const raw = window.localStorage.getItem(SESSION_KEY);
6    return raw ? JSON.parse(raw) : null;
7  }
8
9  export function setStoredSession(session) {
10   window.localStorage.setItem(SESSION_KEY, JSON.stringify(session));
11 }
12
13 export function clearStoredSession() {
14   window.localStorage.removeItem(SESSION_KEY);
15 }
16
17 async function request(path, options = {}) {
18   const session = getStoredSession();
19   const headers = new Headers(options.headers || {});
20
21   if (session?.token) {
22     headers.set("Authorization", `Bearer ${session.token}`);
23   }
24
25   const response = await fetch(`${API_BASE_URL}${path}`, {
26     ...options,
27     headers,
28   });
29   const payload = await response.json().catch(() => ({}));
```

Hình 5: Gọi API backend

3.3 Environment

Dự án tuân thủ nguyên tắc:

- .env không commit lên Git
- .env.example phải commit để người khác có thể cấu hình chạy lại

Trong repo có các file env mẫu:

- .env.example ở cấp expense-tracker/
- .env.example cho backend
- .env.example và .env.production.example cho frontend

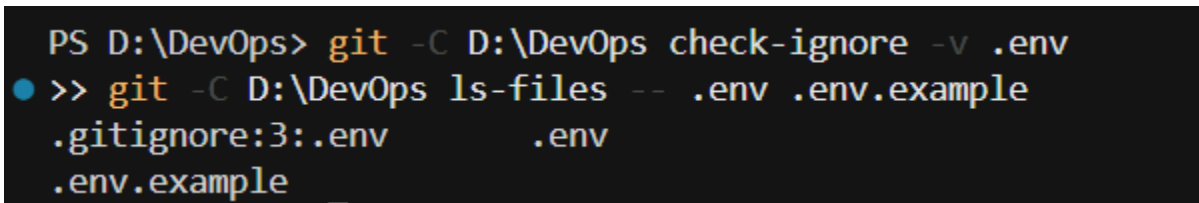
Các biến quan trọng:

- Backend:
 - DB_*, PORT, CLIENT_URL, NODE_ENV
- Frontend:
 - VITE_API_URL (trở về backend)

Cam kết không hardcode

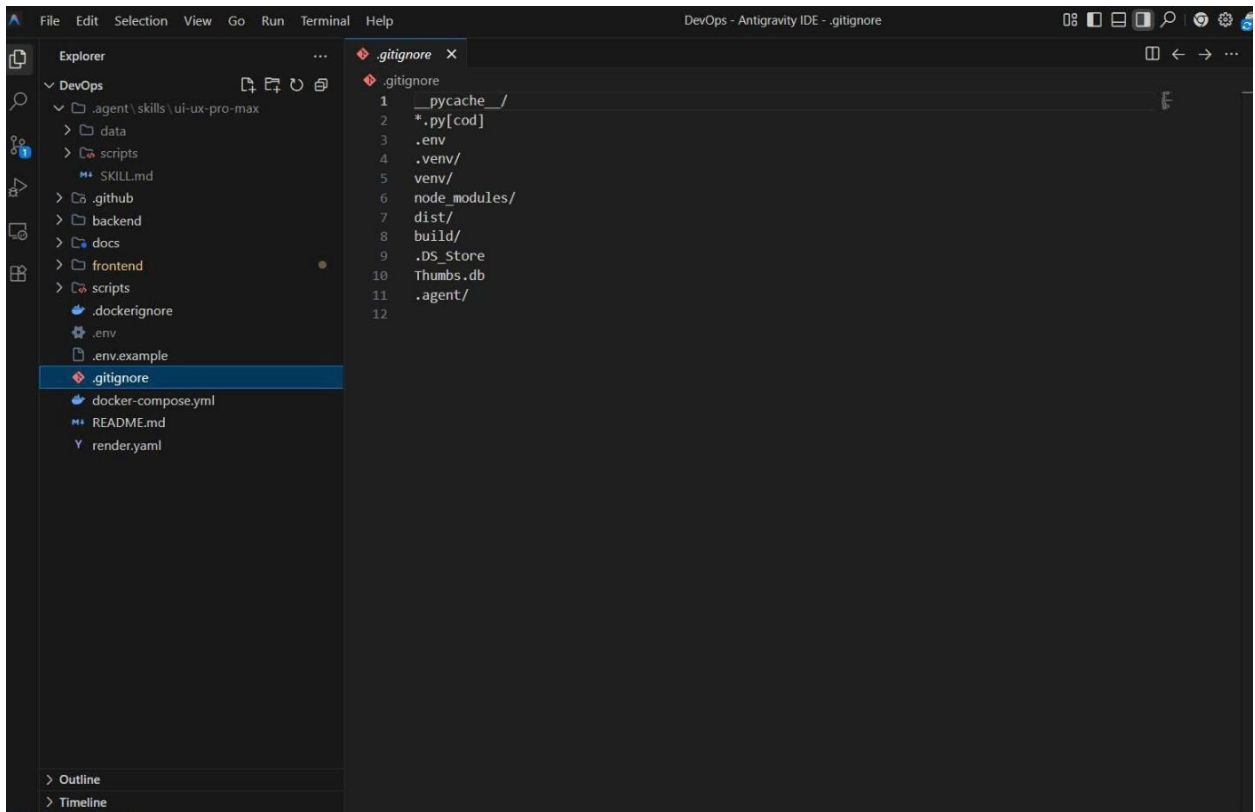
- Không hardcode DB/URL trong code; mọi thứ lấy từ env hoặc default hợp lý.
- Docker Compose truyền env cho backend và build-arg cho frontend.

Mình chứng không commit .env

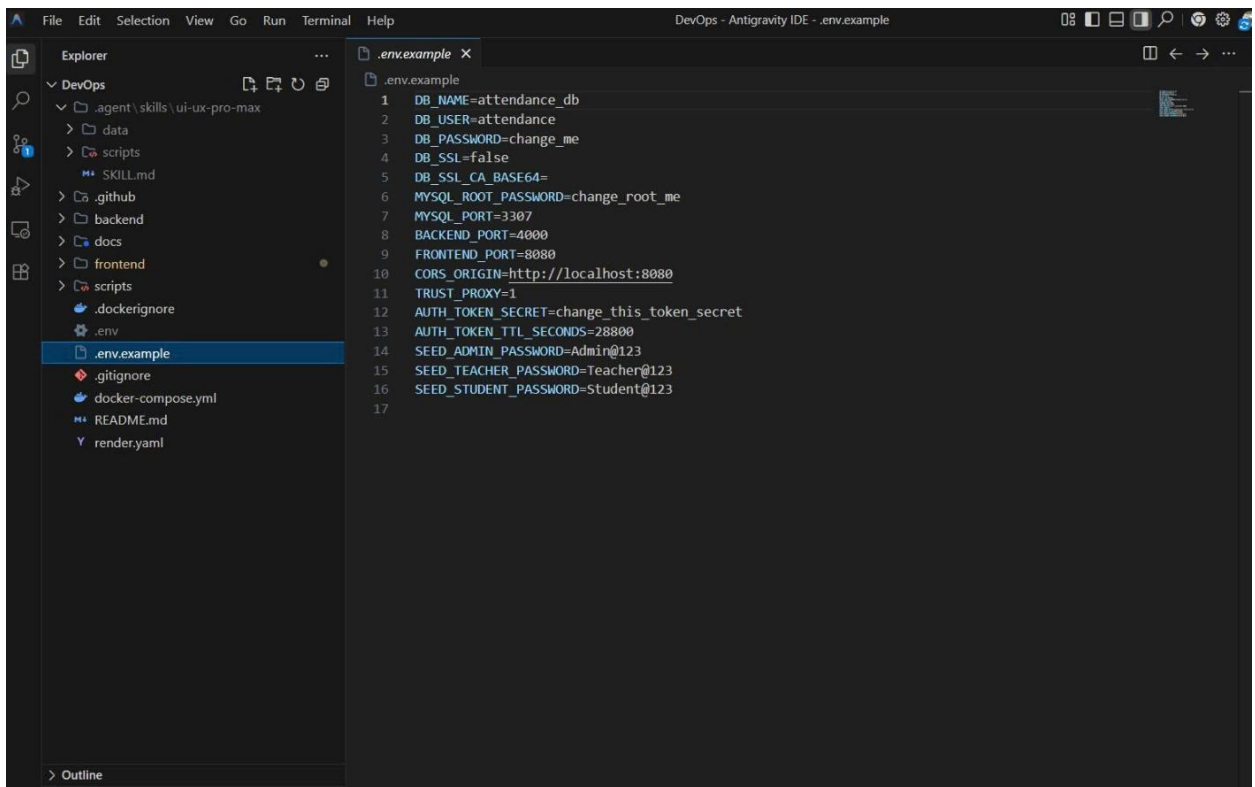


```
PS D:\DevOps> git -C D:\DevOps check-ignore -v .env
● >> git -C D:\DevOps ls-files -- .env .env.example
.gitignore:3:.env .env
.env.example
```

Hình 6 File .env đã được cấu hình để Git bỏ qua



Hình 6: .gittinore / hoặc danh sách file repo không có .env



Hình 7: file .env.example

3.4 Docker

Hệ thống được Docker hoá theo mô hình multi-service bằng docker-compose.yml, gồm 3 services:

1. mysql-db
 - MySQL 8.0
 - Khởi tạo schema tự động thông qua mount schema.sql
 - Có volume lưu dữ liệu
2. backend
 - Build image từ Dockerfile backend
 - Nhận env runtime để kết nối DB và phục vụ API
 - Phụ thuộc vào mysql-db
3. frontend
 - Build frontend thành static assets
 - Serve bằng Nginx (port container 80)
 - Có thể cấu hình gọi backend qua /api hoặc VITE_API_URL

Lệnh chạy bắt buộc

- docker compose up -d

Minh chứng Docker:

```
PS C:\Users\This PC\Desktop\AG\1. Devops\DevOps> docker compose up -d
[+] up 4/4
✓ Container devops-mysql-1   Healthy      0.7s
✓ Container devops-adminer-1 Running      0.0s
✓ Container devops-backend-1 Healthy      1.2s
✓ Container devops-frontend-1 Running       0.0s
PS C:\Users\This PC\Desktop\AG\1. Devops\DevOps>
```

Hình 8: Chạy docker compose up -d thành công

CONTAINER ID	IMAGE	COMMAND	CREATED	STATUS	PORTS	Add to Chat Ctrl+L
7d89a790b2fd	devops-frontend	"/docker-entrypoint..."	32 minutes ago	Up 32 minutes	0.0.0.0:8080->80/tcp, [::]:8080->80/tcp	
baa7e7a4a10b	devops-backend	"docker-entrypoint.s..."	32 minutes ago	Up 32 minutes (healthy)	0.0.0.0:4000->4000/tcp, [::]:4000->4000/t	
cp	devops-backend-1					
d5a3414b708f	adminer:4	"entrypoint.sh docke..."	7 days ago	Up 33 minutes	0.0.0.0:8083->8080/tcp, [::]:8083->8080/t	
cp	devops-adminer-1					
6494006c6224	mysql:8.4	"docker-entrypoint.s..."	7 days ago	Up 33 minutes (healthy)	0.0.0.0:3307->3306/tcp, [::]:3307->3306/t	
cp	devops-mysql-1					

Hình 9: Docker ps hiển thị 3 container running

```

PS D:\DevOps> docker logs devops-backend-1
{"level":"warn","message":"Check constraint 'users_chk_1' is not found in the table.,"time":"2026-05-27T16:59:20.412Z","statement":"ALTER TABLE users DR
OP CHECK users_chk_1"}
{"level":"warn","message":"Duplicate column name 'student_id',"time":"2026-05-27T16:59:20.415Z","statement":"ALTER TABLE users ADD COLUMN student_id INT
NULL"}
{"level":"warn","message":"Check constraint 'attendance_chk_1' is not found in the table.,"time":"2026-05-27T16:59:20.437Z","statement":"ALTER TABLE att
endance DROP CHECK attendance_chk_1"}
{"level":"warn","message":"Duplicate column name 'marked_by_user_id',"time":"2026-05-27T16:59:20.439Z","statement":"ALTER TABLE attendance ADD COLUMN ma
rked_by_user_id INT NULL"}
{"level":"warn","message":"Duplicate column name 'absence_reason',"time":"2026-05-27T16:59:20.441Z","statement":"ALTER TABLE attendance ADD COLUMN absen
ce_reason VARCHAR(255) NULL"}
{"level":"warn","message":"Duplicate column name 'is_excused',"time":"2026-05-27T16:59:20.444Z","statement":"ALTER TABLE attendance ADD COLUMN is_excuse
d BOOLEAN NOT NULL DEFAULT FALSE"}
{"level":"info","message":"database migration completed","time":"2026-05-27T16:59:20.453Z"}
{"level":"info","message":"class schedules seed completed","time":"2026-05-27T16:59:20.665Z","count":4}
{"level":"info","message":"database seed completed","time":"2026-05-27T16:59:20.666Z","users":3,"students":4}
{"level":"info","message":"backend listening","time":"2026-05-27T16:59:20.673Z","port":4000,"environment":"production"}
{"level":"info","message":"request","time":"2026-05-27T16:59:24.755Z","method":"GET","path":"/health","status":200,"ms":8}
{"level":"info","message":"request","time":"2026-05-27T16:59:54.841Z","method":"GET","path":"/health","status":200,"ms":3}
{"level":"info","message":"request","time":"2026-05-27T17:00:18.867Z","method":"GET","path":"/health","status":200,"ms":4}
{"level":"error","message":"request failed","time":"2026-05-27T17:00:18.870Z","method":"GET","path":"/api/classes","statusCode":401,"error":"Token has ex
pired."}
{"level":"info","message":"request","time":"2026-05-27T17:00:18.871Z","method":"GET","path":"/api/classes","status":401,"ms":3}
{"level":"info","message":"request","time":"2026-05-27T17:00:24.910Z","method":"GET","path":"/health","status":200,"ms":2}
{"level":"info","message":"request","time":"2026-05-27T17:00:36.036Z","method":"POST","path":"/auth/login","status":200,"ms":57}
{"level":"info","message":"request","time":"2026-05-27T17:00:36.075Z","method":"GET","path":"/classes","status":304,"ms":13}
{"level":"info","message":"request","time":"2026-05-27T17:00:36.079Z","method":"GET","path":"/health","status":200,"ms":15}
{"level":"info","message":"request","time":"2026-05-27T17:00:36.094Z","method":"GET","path":"/students","status":304,"ms":6}
{"level":"info","message":"request","time":"2026-05-27T17:00:36.115Z","method":"GET","path":"/attendance/dates","status":304,"ms":8}
{"level":"info","message":"request","time":"2026-05-27T17:00:36.124Z","method":"GET","path":"/attendance","status":200,"ms":19}
{"level":"info","message":"request","time":"2026-05-27T17:00:36.139Z","method":"GET","path":"/stats","status":200,"ms":6}

```

Hình 10: logs backend (docker logs devops-backend-1)

```

PS D:\DevOps> docker logs devops-mysql-1
2026-05-20 02:59:21:00:00 [Note] [Entrypoint]: Entrypoint script for MySQL Server 8.4.9-1.el9 started.
2026-05-20 02:59:21:00:00 [Note] [Entrypoint]: Switching to dedicated user 'mysql'
2026-05-20 02:59:21:00:00 [Note] [Entrypoint]: Entrypoint script for MySQL Server 8.4.9-1.el9 started.
2026-05-20 02:59:22:00:00 [Note] [Entrypoint]: Initializing database files
2026-05-20T02:59:22.058051Z 0 [System] [MY-015017] [Server] MySQL Server Initialization - start.
2026-05-20T02:59:22.059859Z 0 [System] [MY-013169] [Server] /usr/sbin/mysqld (mysqld 8.4.9) initializing of server in progress as process 80
2026-05-20T02:59:22.071052Z 1 [System] [MY-013576] [InnoDB] InnoDB initialization has started.
2026-05-20T02:59:22.538900Z 1 [System] [MY-013577] [InnoDB] InnoDB initialization has ended.
2026-05-20T02:59:24.120973Z 6 [Warning] [MY-010453] [Server] root@localhost is created with an empty password ! Please consider switching off the --initia
lize-insecure option.
2026-05-20T02:59:26.505902Z 0 [System] [MY-015018] [Server] MySQL Server Initialization - end.
2026-05-20 02:59:26:00:00 [Note] [Entrypoint]: Database files initialized
2026-05-20 02:59:26:00:00 [Note] [Entrypoint]: Starting temporary server
2026-05-20T02:59:26.586858Z 0 [System] [MY-015015] [Server] MySQL Server - start.
2026-05-20T02:59:26.924886Z 0 [System] [MY-010116] [Server] /usr/sbin/mysqld (mysqld 8.4.9) starting as process 131
2026-05-20T02:59:26.953290Z 1 [System] [MY-013576] [InnoDB] InnoDB initialization has started.
2026-05-20T02:59:27.447538Z 1 [System] [MY-013577] [InnoDB] InnoDB initialization has ended.
2026-05-20T02:59:27.938378Z 0 [Warning] [MY-010068] [Server] CA certificate ca.pem is self signed.
2026-05-20T02:59:27.938456Z 0 [System] [MY-013602] [Server] Channel mysql_main configured to support TLS. Encrypted connections are now supported for thi
s channel.
2026-05-20T02:59:27.944138Z 0 [Warning] [MY-011810] [Server] Insecure configuration for --pid-file: Location '/var/run/mysqld' in the path is accessible
to all OS users. Consider choosing a different directory.
2026-05-20T02:59:27.987173Z 0 [System] [MY-011323] [Server] X Plugin ready for connections. Socket: /var/run/mysqld/mysqld.sock
2026-05-20T02:59:27.987513Z 0 [System] [MY-010931] [Server] /usr/sbin/mysqld: ready for connections. Version: '8.4.9' socket: '/var/run/mysqld/mysqld.so
ck' port: 0 MySQL Community Server - GPL.
2026-05-20 02:59:28:00:00 [Note] [Entrypoint]: Temporary server started.
'/var/lib/mysql/mysql.sock' -> '/var/run/mysqld/mysqld.sock'
Warning: Unable to load '/usr/share/zoneinfo/iso3166.tab' as time zone. Skipping it.
Warning: Unable to load '/usr/share/zoneinfo/leap-seconds.list' as time zone. Skipping it.
Warning: Unable to load '/usr/share/zoneinfo/leapseconds' as time zone. Skipping it.
Warning: Unable to load '/usr/share/zoneinfo/tzdata.zi' as time zone. Skipping it.

```

Hình 11: logs mysql (docker logs expense_mysql)

```

PS D:\DevOps> docker logs devops-frontend-1
/docker-entrypoint.sh: /docker-entrypoint.d/ is not empty, will attempt to perform configuration
/docker-entrypoint.sh: Looking for shell scripts in /docker-entrypoint.d/
/docker-entrypoint.sh: Launching /docker-entrypoint.d/10-listen-on-ipv6-by-default.sh
10-listen-on-ipv6-by-default.sh: info: Getting the checksum of /etc/nginx/conf.d/default.conf
10-listen-on-ipv6-by-default.sh: info: /etc/nginx/conf.d/default.conf differs from the packaged version
/docker-entrypoint.sh: Sourcing /docker-entrypoint.d/15-local-resolvers.envsh
/docker-entrypoint.sh: Launching /docker-entrypoint.d/20-envsubst-on-templates.sh
/docker-entrypoint.sh: Launching /docker-entrypoint.d/30-tune-worker-processes.sh
/docker-entrypoint.sh: Configuration complete; ready for start up
2026/05/27 16:59:27 [notice] 1#1: using the "epoll" event method
2026/05/27 16:59:27 [notice] 1#1: nginx/1.27.5
2026/05/27 16:59:27 [notice] 1#1: built by gcc 14.2.0 (Alpine 14.2.0)
2026/05/27 16:59:27 [notice] 1#1: OS: Linux 6.6.87.2-microsoft-standard-WSL2
2026/05/27 16:59:27 [notice] 1#1: getrlimit(RLIMIT_NOFILE): 1048576:1048576
2026/05/27 16:59:27 [notice] 1#1: start worker processes
2026/05/27 16:59:27 [notice] 1#1: start worker process 30
2026/05/27 16:59:27 [notice] 1#1: start worker process 31
2026/05/27 16:59:27 [notice] 1#1: start worker process 32
2026/05/27 16:59:27 [notice] 1#1: start worker process 33
2026/05/27 16:59:27 [notice] 1#1: start worker process 34
2026/05/27 16:59:27 [notice] 1#1: start worker process 35
2026/05/27 16:59:27 [notice] 1#1: start worker process 36
2026/05/27 16:59:27 [notice] 1#1: start worker process 37
2026/05/27 16:59:27 [notice] 1#1: start worker process 38
2026/05/27 16:59:27 [notice] 1#1: start worker process 39
2026/05/27 16:59:27 [notice] 1#1: start worker process 40
2026/05/27 16:59:27 [notice] 1#1: start worker process 41
2026/05/27 16:59:27 [notice] 1#1: start worker process 42
2026/05/27 16:59:27 [notice] 1#1: start worker process 43
2026/05/27 16:59:27 [notice] 1#1: start worker process 44
2026/05/27 16:59:27 [notice] 1#1: start worker process 45
172.20.0.1 - - [27/May/2026:17:00:18 +0000] "GET / HTTP/1.1" 200 488 "-" "Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/148.0.0.0 Safari/537.36" "-"
172.20.0.1 - - [27/May/2026:17:00:18 +0000] "GET /assets/index-VuUd4T-D.css HTTP/1.1" 200 7482 "http://localhost:8080/" "Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/148.0.0.0 Safari/537.36" "-"
172.20.0.1 - - [27/May/2026:17:00:18 +0000] "GET /assets/index-BWdW99v.js HTTP/1.1" 200 59580 "http://localhost:8080/" "Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/148.0.0.0 Safari/537.36" "-"

```

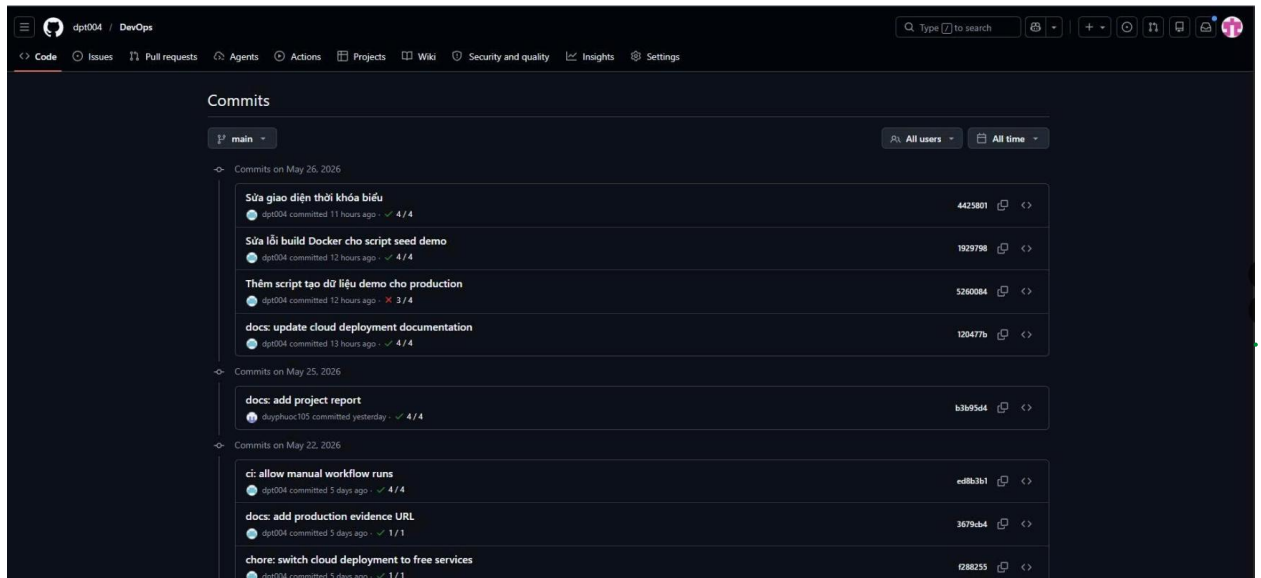
Hình 12: logs frontend (docker logs expense_frontend)

3.5 Git & Branching

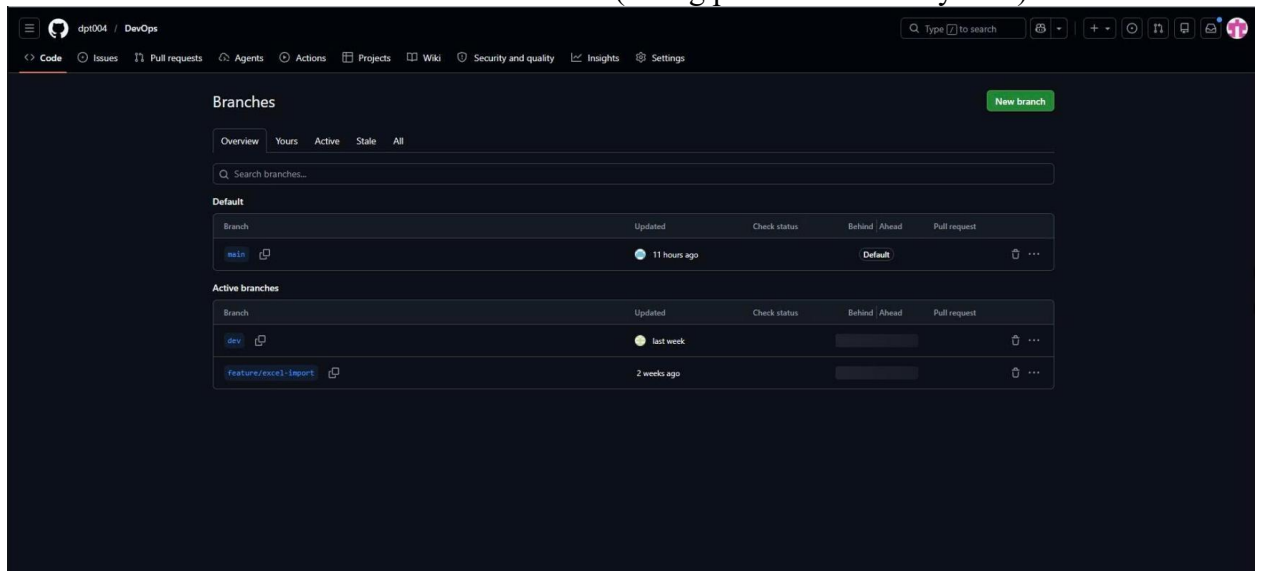
Dự án tổ chức nhánh theo hướng làm việc nhóm:

- Có nhánh main để tổng hợp phiên bản ổn định.
- Có nhánh phát triển develop (vai trò tương đương nhánh dev trong yêu cầu).
- Có nhiều nhánh dạng feature như feature-* phục vụ phát triển tính năng độc lập.
- Lịch sử commit có nhiều lần cập nhật, thể hiện quá trình phát triển và sửa lỗi (CI/CD, Docker, deploy config...).

Mình chứng lịch sử commit



Hình 13: Danh sách commit trên GitHub (không phải 1 commit duy nhất)



Hình 14: danh sách branch hiển thị main/develop/feature-*

3.6 CI — Continuous Integration

Hệ thống dùng GitHub Actions để tự động kiểm tra trước khi tích hợp code.
Yêu cầu pipeline

- install
- lint
- test
- build

Mô hình chạy

- Backend job:
 - npm install
 - npm run lint
 - npm test
- Frontend job:
 - npm install --legacy-peer-deps
 - fix permissions (đảm bảo runner chạy được các binary)
 - npm run lint
 - npm test
 - npm run build

Pipeline được thiết kế để fail ngay khi có lỗi lint/test/build, đảm bảo không “bypass”.
 Minh chứng CI pass

Hình 15: workflow run pass trên GitHub Actions

```
Windows PowerShell
PS C:\Users\This PC\Desktop\AG\1. Devops\DevOps> cd frontend; npm ci
added 66 packages, and audited 67 packages in 6s
9 packages are looking for funding
  run 'npm fund' for details
found 0 vulnerabilities
PS C:\Users\This PC\Desktop\AG\1. Devops\DevOps\frontend> npm run lint
> attendance-frontend@0.1.0 lint
> node scripts/lint.mjs src tests scripts

Lint passed.
PS C:\Users\This PC\Desktop\AG\1. Devops\DevOps\frontend> npm test
> attendance-frontend@0.1.0 test
> node --test

✓todayISO returns yyyy-mm-dd in local time (1.9337ms)
✓startOfMonthISO returns the first day of the current month (0.2169ms)
✓isoDayOfWeek uses Monday=1 .. Sunday=7 (0.1546ms)
✓dateForWeekday returns ISO date in current week (0.1479ms)
Tests 4
Suites 0
Pass 4
Fail 0
Cancelled 0
Skipped 0
Node 0
Duration_ms 188.2889
PS C:\Users\This PC\Desktop\AG\1. Devops\DevOps\frontend> npm run build
> attendance-frontend@0.1.0 build
> vite build

vite v6.4.2 building for production...
✓ 41 modules transformed.
dist/index.html      0.85 kB | gzip: 0.48 kB
dist/assets/index-YwUd4T-D.css  38.04 kB | gzip: 7.47 kB
dist/assets/index-BWxNN99v.js  196.75 kB | gzip: 59.58 kB
✓ built in 784ms
PS C:\Users\This PC\Desktop\AG\1. Devops\DevOps\frontend> |
```

```
Windows PowerShell
found 0 vulnerabilities
PS C:\Users\This PC\Desktop\AG\1. Devops\DevOps\frontend> npm ci

added 66 packages, and audited 67 packages in 4s

9 packages are looking for funding
  run `npm fund` for details

found 0 vulnerabilities
PS C:\Users\This PC\Desktop\AG\1. Devops\DevOps\frontend> npm run lint

> attendance-frontend@0.1.0 lint
> node scripts/lint.mjs src tests scripts

Lint passed.
PS C:\Users\This PC\Desktop\AG\1. Devops\DevOps\frontend> npm test

> attendance-frontend@0.1.0 test
> node --test

✓todayISO returns yyyy-mm-dd in local time (2.7158ms)
✓startOfMonthISO returns the first day of the current month (0.1992ms)
✓isoDayOfWeek uses Monday=1 .. Sunday=7 (0.1414ms)
✓dateForWeekday returns ISO date in current week (0.1483ms)
Tests 4
Suites 0
Pass 4
Fail 0
Cancelled 0
Skipped 0
Node 0
Duration_ms 115.6833
PS C:\Users\This PC\Desktop\AG\1. Devops\DevOps\frontend> npm run build

> attendance-frontend@0.1.0 build
> vite build

vite v6.4.2 building for production...
✓ 41 modules transformed.
dist/index.html      0.85 kB | gzip: 0.48 kB
dist/assets/index-YwUd4T-D.css 38.04 kB | gzip: 7.47 kB
dist/assets/index-BWxNN99v.js 196.75 kB | gzip: 59.58 kB
✓ built in 805ms
PS C:\Users\This PC\Desktop\AG\1. Devops\DevOps\frontend> |
```

Hình 16: chi tiết từng bước install/lint/test/build

CHƯƠNG 4: DEPLOY

4.1 Môi trường

Dự án chuẩn bị sẵn nhiều phương án deploy:

- Render: deploy backend Node service bằng render.yaml
- Vercel: deploy frontend React/Vite (có cấu hình trong thư mục frontend)

4.2 Quy trình deploy

Bước 1: Deploy backend

- Tạo service trên Render, kết nối repository.
- Render đọc cấu hình render.yaml, build và start backend.
- Khai báo env DB_* và CLIENT_URL trên dashboard Render.

Bước 2: Deploy frontend

- Import project lên Vercel.
- Set env VITE_API_URL trỏ tới URL backend Render.
- Deploy và lấy URL public.

Bước 3: Kiểm tra sau deploy

- Kiểm tra backend: /api/health
- Kiểm tra frontend load, thao tác CRUD, không lỗi console.

4.3 Minh chứng

- URL public:
 - Backend:
 - Frontend:

Screenshot hệ thống chạy online

Hình 15: Trang frontend dashboard online

CHƯƠNG 5: LOGGING & DEBUG

5.1 Logging

Hệ thống triển khai logging ở nhiều cấp:

Backend log

- Dùng morgan('dev') để log request (method, path, status, time).

Docker log

- Khi chạy container, toàn bộ stdout/stderr của service có thể xem bằng docker logs.
- Đây là nguồn log quan trọng khi debug production.

Deploy log

- Render cung cấp build log và runtime log.
- Vercel cung cấp build log, và log theo môi trường (tùy cấu hình).

5.2 Debug theo layer

Để đảm bảo khả năng vận hành và xử lý sự cố, quá trình debug được chia theo layer:

L4: Frontend

- Kiểm tra Console (lỗi JS, lỗi CORS).
- Kiểm tra Network: request có gửi đúng endpoint không, status code.
- Kiểm tra env VITE_API_URL và cấu hình build.

L3: Backend

- Kiểm tra log morgan để biết request có vào server hay không.

- Kiểm tra error handler (500/400).
- Test nhanh bằng /api/health và các endpoint CRUD.

L2: Database

- Kiểm tra MySQL container hoặc MySQL service chạy chưa.
- Kiểm tra schema đã init chưa.
- Test query và kiểm tra quyền truy cập.

L1: Infrastructure

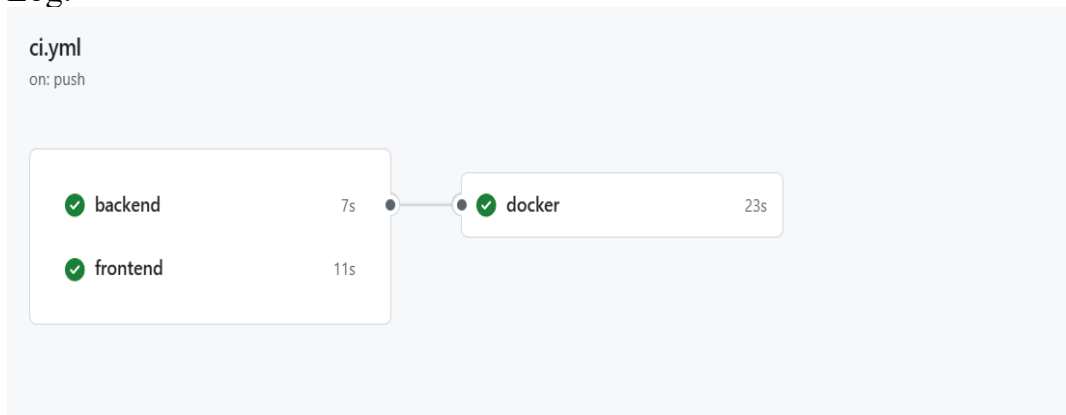
- Kiểm tra port mapping, firewall, domain/HTTPS, network Docker.
- Kiểm tra biến môi trường trên Render/Vercel.
- Kiểm tra service restart policy, health status.

5.3. INCIDENT

Lưu ý: Bạn nên thay phần “Log” và “Hình minh họa” bằng log/ảnh chụp thật khi bạn chạy và gặp lỗi.

Incident 1:

- Hiện tượng: CI frontend fail, không chạy được một số lệnh build/test do lỗi permission của binary trong node_modules/.bin.
- Log:



Hình 17 GITHUB ACTIONS

- Layer: L3 (CI environment/build)
- Nguyên nhân: Quyền thực thi của file binary trong môi trường runner Linux không đúng hoặc bị hạn chế.
- Cách fix: Thêm bước cấp quyền:
 - `chmod -R +x node_modules/.bin`
- Cách phòng tránh: Chuẩn hoá lệnh install (ưu tiên npm ci nếu phù hợp), kiểm tra pipeline trên Linux từ sớm.

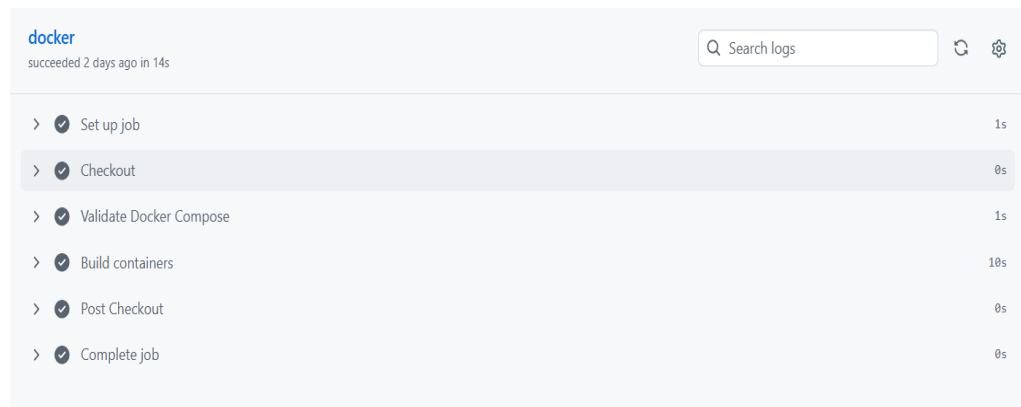
- Hình minh họa:

```

1 ▶ Run docker compose build
4 #1 [internal] load local bake definitions
5 #1 reading from stdin 684B done
6 #1 DONE 0.0s
7
8 #2 [frontend internal] load build definition from Dockerfile
9 #2 transferring dockerfile: 419B 0.0s done
10 #2 DONE 0.0s
11
12 #3 [backend internal] load build definition from Dockerfile
13 #3 transferring dockerfile: 522B 0.0s done
14 #3 DONE 0.0s
15
16 #4 [auth] library/nginx:pull token for registry-1.docker.io
17 #4 DONE 0.0s
18
19 #5 [auth] library/node:pull token for registry-1.docker.io
20 #5 DONE 0.0s

```

Hình 18: Step fail



Hình 19: Pass sau khi fix

Incident 2:

- Hiện tượng: Test tooling (ví dụ vitest) báo lỗi không tương thích phiên bản Node trong CI/Docker.
- Log: Sau khi chuẩn hóa test tooling về `node --test`, frontend pass 4/4 tests và backend pass 13/13 tests; không còn lỗi không tương thích Vitest/Node trong CI/Docker.
- Layer: L3 (CI/build runtime)
- Nguyên nhân: Node version quá thấp so với yêu cầu dependencies.
- Cách fix: Nâng Node lên version 20 trong CI và Docker.
- Cách phòng tránh: Pin Node version nhất quán (local/CI/Docker), kiểm tra “engines” của package.
- Hình minh họa: [CHÈN ẢNH: log lỗi + cấu hình node-version 20]

```

✓hashPassword is deterministic for the same username salt (82.1888ms)
✓createToken and verifyToken preserve role payload (0.7174ms)
✓normalizeStudentRows accepts Vietnamese Excel headers (1.6971ms)
✓normalizeStudentRows de-duplicates by student code (0.3105ms)
✓normalizeStudentRows can force one class for the whole import file (0.2214ms)
✓timeToMinutes parses HH:MM (1.2866ms)
✓normalizeTime pads single-digit hour (0.2546ms)
✓rangesOverlap detects overlapping intervals (0.1395ms)
✓rangesOverlap rejects invalid end before start at validation layer (0.5124ms)
✓normalizeDate accepts ISO calendar dates (1.9727ms)
✓normalizeDate rejects invalid dates (1.0476ms)
✓normalizeStatus only accepts supported attendance states (0.3289ms)
✓validateAttendanceRecords normalizes valid records (1.043ms)
i tests 13
i suites 0
i pass 13
i fail 0
i cancelled 0
i skipped 0
i todo 0
i duration_ms 331.6161

```

Hình 20: npm.cmd test ở Backend

```

● PS D:\DevOps\Frontend> npm.cmd test

> attendance-frontend@0.1.0 test
> node --test

✓todayISO returns yyyy-mm-dd in local time (1.8403ms)
✓startOfMonthISO returns the first day of the current month (0.2208ms)
✓isoDayOfWeek uses Monday=1 ... Sunday=7 (0.1275ms)
✓dateForWeekday returns ISO date in current week (0.1723ms)
i tests 4
i suites 0
i pass 4
i fail 0
i cancelled 0
i skipped 0
i todo 0
i duration_ms 114.3044

```

Hình 21: npm.cmd test ở Fronted

Incident 3:

- Hiện tượng: Deploy backend trên Render fail do cấu hình build/start hoặc thiếu env DB_.*.
- Log:

```
{"level":"error","message":"backend failed to start","time":"2026-05-27T18:12:18.393Z","error":""}
```

AggregateError:

```
at query (file:///D:/DevOps/backend/src/db/pool.js:56:34)
```

```
at Module.migrate (file:///D:/DevOps/backend/src/db/schema.js:39:9) {
```

```
code: 'ECONNREFUSED',
```

```
fatal: true,
```

```
[errors]: [
```

```
  Error: connect ECONNREFUSED ::1:3306
```

```
  Error: connect ECONNREFUSED 127.0.0.1:3306
```

```
]
```

```
}
```

- Layer: L1 (deploy platform) + L3 (backend runtime)
- Nguyên nhân: rootDir/buildCommand/startCommand chưa đúng hoặc env chưa được khai báo trên dashboard.
- Cách fix: Sử dụng render.yaml chuẩn hoá cấu hình và khai báo biến môi trường sync:false để nhập thủ công trên Render.
- Cách phòng tránh: Luôn có file cấu hình deploy, checklist env trước khi deploy; test production mode local.
- Hình minh họa:

✖	01:19:47 AM	[76gmg]	{"level":"error","message":"backend failed to start","time":"2026-05-27T18:19:47.600Z","error":""}
ℹ	01:24:46 AM		==> Deploying...
	01:24:46 AM		==> Setting WEB_CONCURRENCY=1 by default, based on available CPUs in the instance
✖	01:24:54 AM	[jpxtx]	{"level":"error","message":"backend failed to start","time":"2026-05-27T18:24:54.515Z","error":""}
	01:24:57 AM		==> Exited with status 1
	01:24:57 AM		==> Common ways to troubleshoot your deploy: https://render.com/docs/troubleshooting-deploys
✖	01:25:01 AM	[jpxtx]	{"level":"error","message":"backend failed to start","time":"2026-05-27T18:25:01.769Z","error":""}
✖	01:25:19 AM	[jpxtx]	{"level":"error","message":"backend failed to start","time":"2026-05-27T18:25:19.885Z","error":""}

Hình 22: Render build log

May 28	
01:24:24 AM	==> Cloning from https://github.com/dpt004/DevOps
01:24:30 AM	==> Checking out commit 5a29f738fff042e2b454a94b85cdba6fc462d0a4 in branch main
01:24:31 AM	#1 [internal] load build definition from Dockerfile
01:24:31 AM	#1 transferring dockerfile: 499B done
01:24:31 AM	#1 DONE 0.0s
01:24:31 AM	#2 [internal] load metadata for docker.io/library/node:22-alpine
01:24:32 AM	#2 ...
01:24:32 AM	#3 [auth] library/node:pull render-prod/docker-mirror-repository/library/node:pull token for us-west1-docker.pkg.dev
01:24:32 AM	#3 DONE 0.0s
01:24:33 AM	#2 [internal] load metadata for docker.io/library/node:22-alpine
01:24:37 AM	#2 DONE 5.9s
01:24:37 AM	#4 [internal] load .dockerignore

Hình 23: Render build log

CHƯƠNG 6: KẾT QUẢ

6.1 Mô tả hệ thống

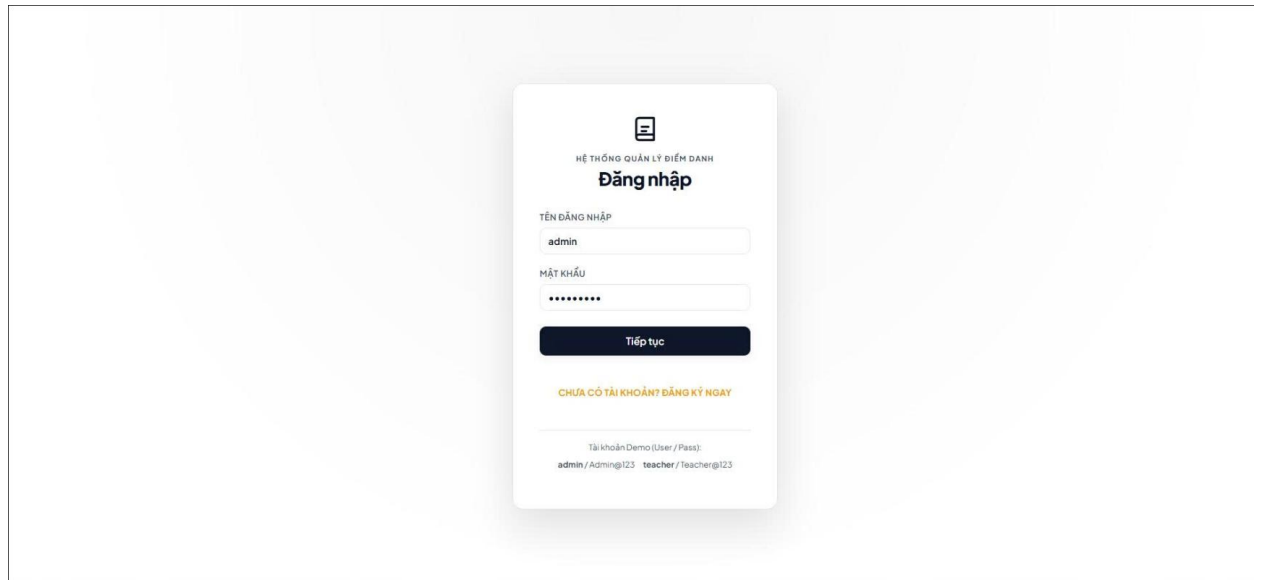
Sau khi hoàn thành, hệ thống quản lý điểm danh đáp ứng đầy đủ các yêu cầu về:

- Kiến trúc FE/BE/DB rõ ràng
- REST API phục vụ CRUD và thống kê dữ liệu điểm danh
- Có health check theo yêu cầu
- Có thể chạy local bằng npm hoặc triển khai nhanh bằng Docker
- Có CI pipeline tự động kiểm tra chất lượng source code
- Có khả năng triển khai thực tế lên Render/Vercel/VPS
- Có logging và debug hỗ trợ quá trình vận hành
- Quản lý cấu hình bằng environment variables (.env)

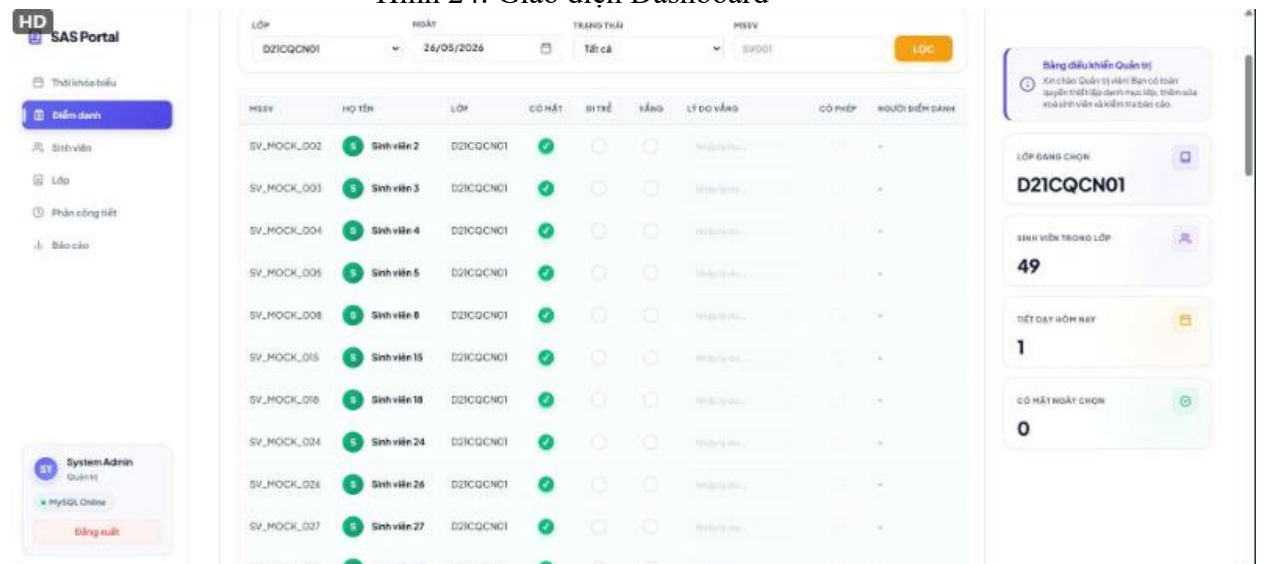
6.2 Minh họa

Bắt buộc:

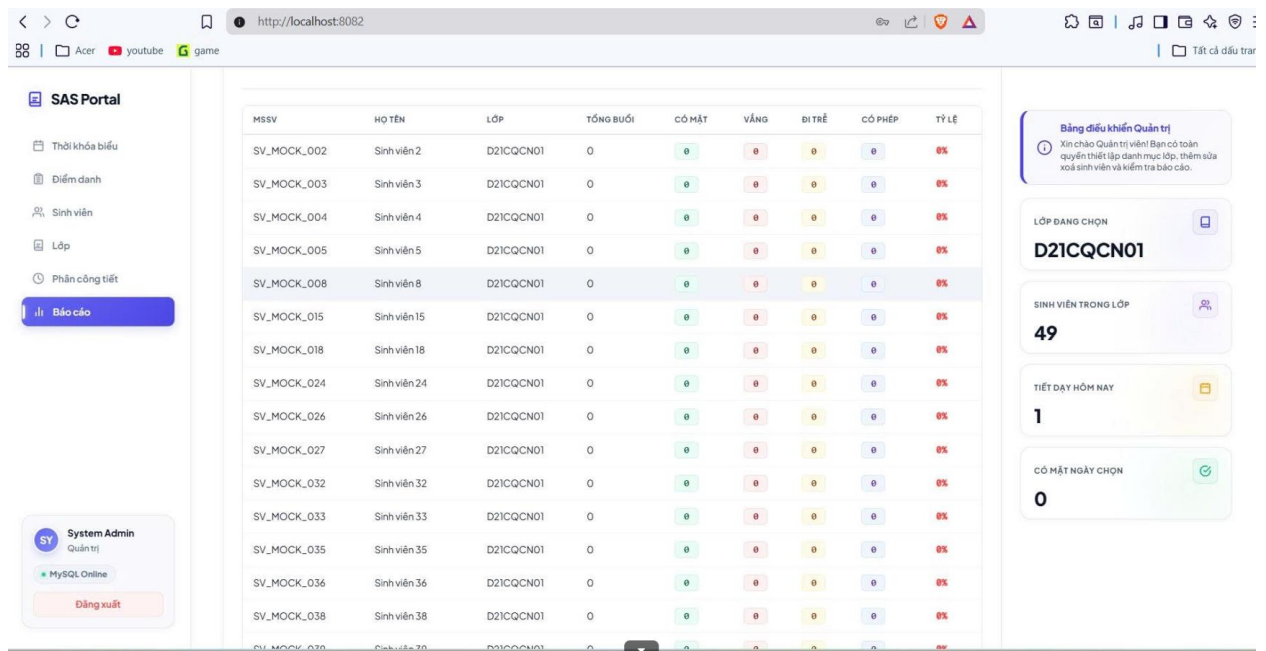
- UI



Hình 24: Giao diện Dashboard

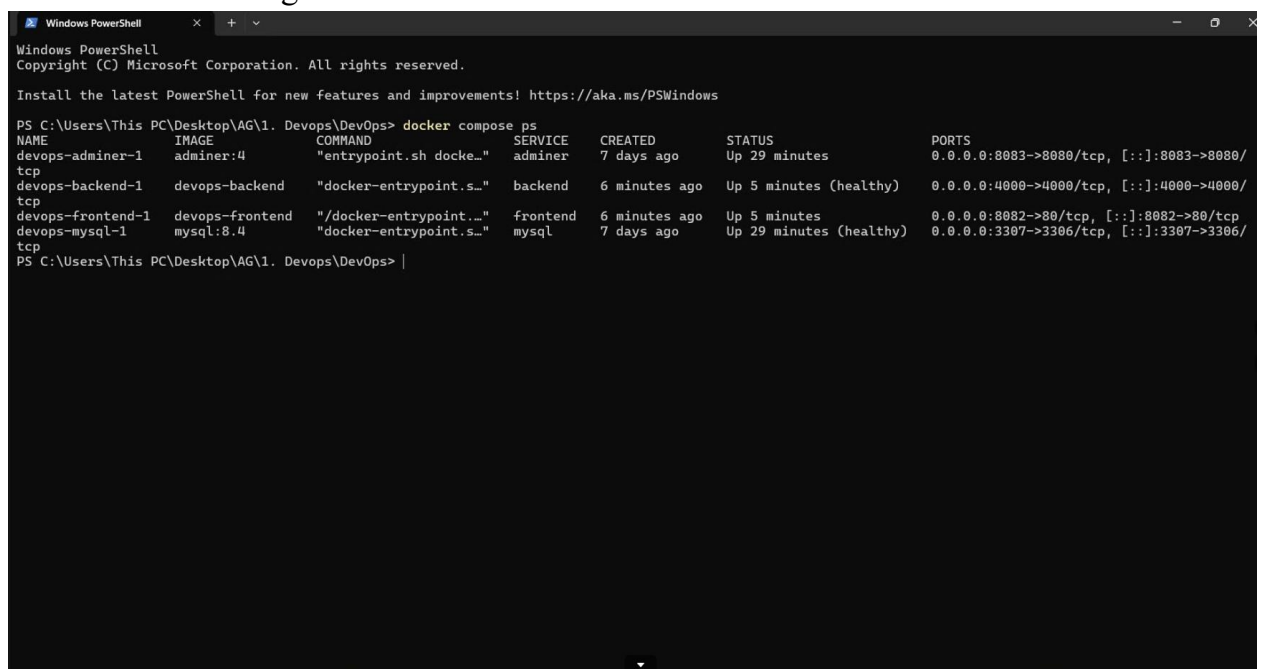


Hình 25: Giao diện bảng điểm danh



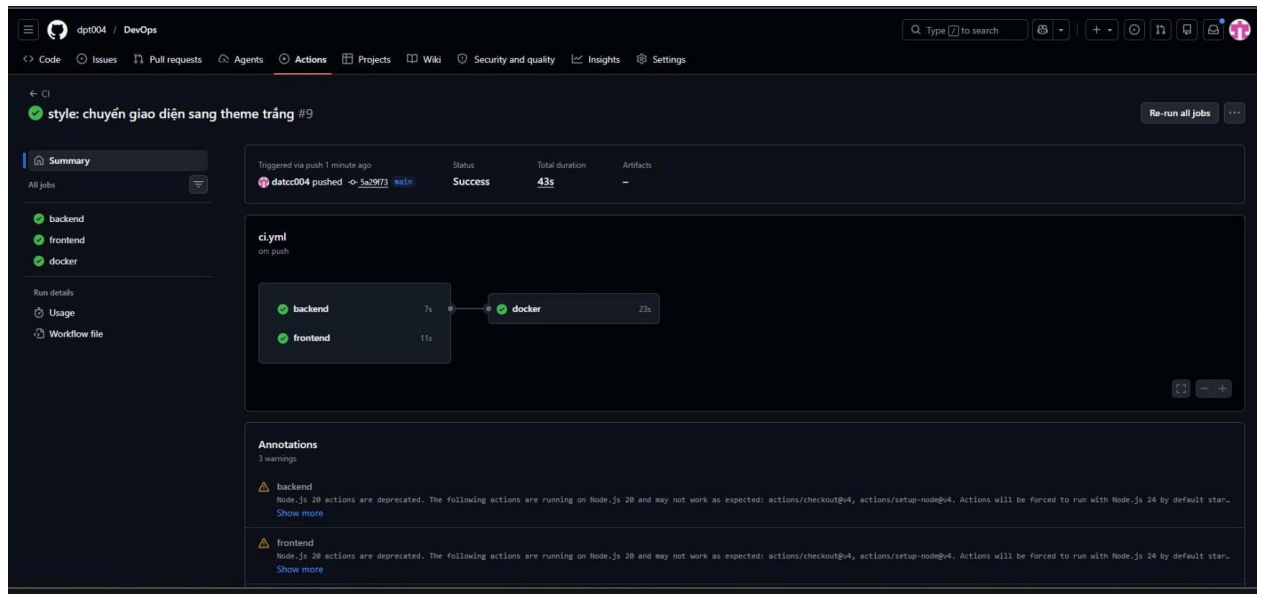
Hình 26: Giao diện báo cáo thống kê

- Docker running



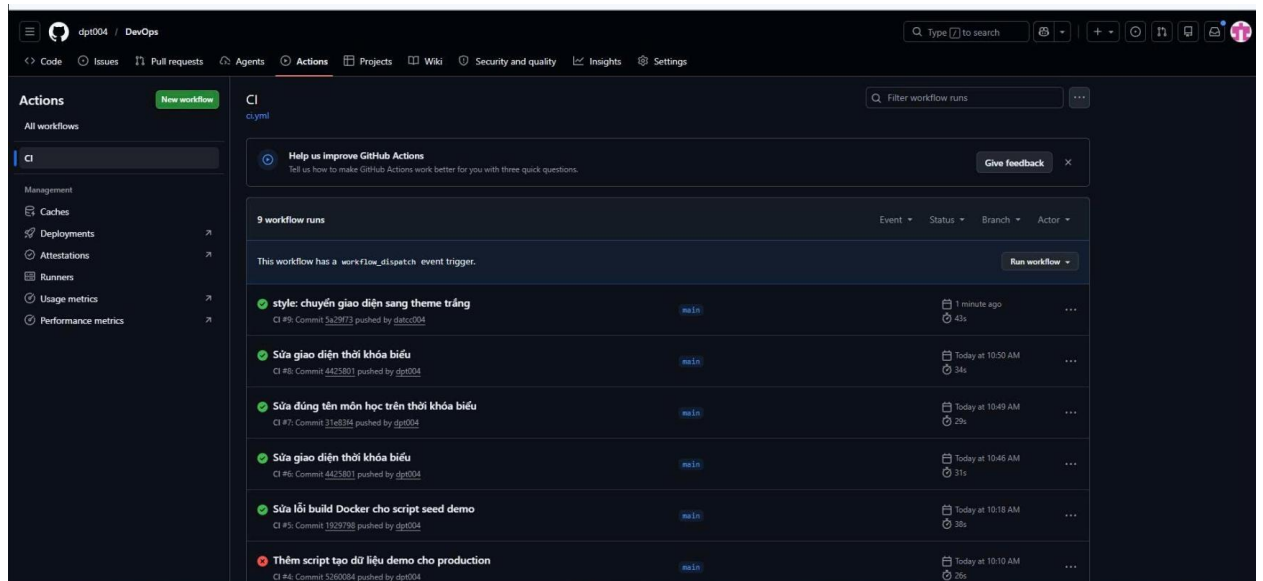
Hình 27: docker ps

- CI/CD pass



Hình 28: GitHub Actions pass

- Deploy thành công



Hình 29: Frontend online

6.3 Đánh giá

Điểm mạnh

- Hệ thống quản lý điểm danh được triển khai theo hướng thực tế với Docker, CI/CD và cấu hình deploy production
- Kiến trúc Frontend / Backend / Database được tách biệt rõ ràng, giúp dễ bảo trì và mở rộng.
- Hệ thống hỗ trợ quản lý sinh viên, điểm danh và thống kê chuyên cần hiệu quả.

- Cấu hình environment rõ ràng thông qua .env, hạn chế hardcode thông tin nhạy cảm.
- Có logging và debug theo từng layer giúp dễ theo dõi và xử lý lỗi.
- Hỗ trợ triển khai nhanh bằng Docker Compose và GitHub Actions.

Điểm hạn chế

- Phần minh chứng cần chạy thật để chụp đủ theo checklist.
- Chưa tích hợp các chức năng nâng cao như xác thực người dùng hoặc phân quyền.
- Tên nhánh phát triển dùng develop (tương đương dev), nếu giảng viên yêu cầu đúng tên dev thì cần tạo thêm nhánh.

CHƯƠNG 7: KẾT LUẬN

7.1 Kết quả đạt được

Đề tài đã hoàn thành một hệ thống quản lý chi tiêu đáp ứng các tiêu chí môn học:

- Chạy được production (có cấu hình deploy + env tách bạch)
- Deploy lại được (có .env.example, docker-compose, CI)
- Debug được theo layer (có logging, có health check, có docker logs/deploy logs)

7.2 Ưu điểm / Hạn chế

Ưu điểm

- Hệ thống quản lý điểm danh được xây dựng theo mô hình Fullstack rõ ràng gồm Frontend / Backend / Database.
- Có Docker và Docker Compose giúp triển khai hệ thống nhanh chóng và đồng nhất giữa các môi trường.
- Có CI/CD pipeline đầy đủ các bước lint, test, build và deploy.
- Không hardcode cấu hình nhạy cảm, toàn bộ thông tin cấu hình được quản lý bằng .env.
- Hệ thống hỗ trợ logging và debug giúp dễ theo dõi request và xử lý lỗi trong quá trình vận hành.
- Có khả năng triển khai thực tế trên môi trường production như Render hoặc Vercel.
- Giao diện quản lý điểm danh trực quan, hỗ trợ quản lý sinh viên và thống kê chuyên cần hiệu quả.

Hạn chế

- Cần bổ sung đầy đủ minh chứng hình ảnh và URL deploy để hoàn thiện theo checklist môn học.
- Chưa tích hợp các chức năng nâng cao như xác thực tài khoản hoặc phân quyền người dùng.
- Chưa hỗ trợ ứng dụng trên thiết bị di động.
- Chức năng báo cáo và thống kê vẫn còn ở mức cơ bản.
- Cần chuẩn hóa mô hình branch để phù hợp hoàn toàn với yêu cầu main/dev/feature/*.

CHECKLIST

- ✓ Frontend load OK
- ✓ Không lỗi console
- ✓ API /api/health OK
- ✓ Docker chạy OK
- ✓ Container running
- ✓ CI/CD pass
- ✓ Deploy có URL
- ✓ Không hardcode config
- ✓ Có ≥ 3 incident